MURU-BENCH: A Benchmark for Mathematical Reasoning Under Uncertainty

Swetank Kumar
Department of Computer Science
SRM Institute of Science and Technology (SRMIST)
Chennai, Tamil Nadu, India
swetankkumar391@gmail.com
https://github.com/swetank18

Abstract

Mathematical reasoning benchmarks have consistently treated correctness as deterministic: a model either produces $3/7$ or it does not. Real-world mathematical reasoning, however, is saturated with uncertainty—epidemiologists work with prevalence *ranges*, engineers propagate parameter uncertainty through failure-rate chains, and decision-makers weigh actions against uncertain utilities. A benchmark that scores only the point estimate, and ignores whether a model knows when it does not know, cannot distinguish a calibrated reasoner from a confident guesser. We introduce MURU-BENCH (**M**athematical **R**easoning **U**nder **U**ncertainty **Bench**mark), a 3,000-problem benchmark in which every prediction must include (i) a point estimate, (ii) a calibrated confidence interval, (iii) a stated confidence probability, and (iv) an identified reasoning framework. Problems span five categories—Bayesian Updating, Conditional Probability Chains, Distribution Estimation, Decision Under Uncertainty, and Adversarial Ambiguity—stratified across five difficulty levels via 21 parametric generation templates with closed-form ground truth, and are partitioned into a 2,398-problem train split, a 301-problem validation split, and a 301-problem held-out test split using stratified sampling on the (category, difficulty) cross-product. The accompanying evaluation harness reports six metrics including Expected Calibration Error and a safety-relevant overconfidence rate, and ships a unified API client for OpenAI, Anthropic, Google, Groq, OpenRouter, and Together AI. We first validate the harness with five simulated capability tiers spanning a 70-percentage-point accuracy range and establish, via paired McNemar tests ($p < 10^{-3}$ for every adjacent-tier comparison) and percentile bootstrap 95% confidence intervals on every metric ($B = 10{,}000$), that the test split has the statistical power to discriminate capability differences as small as 11.6 percentage points. We then report the first language-model leaderboard on MURU-BENCH, evaluating five hosted open-weights models—Llama-3.1-8B, Llama-3.3-70B, Llama-4-Scout-17B, GPT-OSS-120B, and Qwen3-32B—and observe a 51-percentage-point spread in Accuracy@CI on the test set, ranging from 43.1% (Llama-3.1-8B) to 94.5% (GPT-OSS-120B, partial coverage). Across the panel we find a strong negative rank coupling between Accuracy@CI and Expected Calibration Error (Spearman $\rho = -0.90$, Pearson $r = -0.99$): the lowest-accuracy model exhibits an overconfidence rate of 51.4%, while the highest-accuracy models stay below 6%, indicating that miscalibration on this benchmark is dominated by capability rather than by an independent calibration deficit. The benchmark, evaluation harness, generation templates, statistical scripts, raw model responses, tests, and continuous-integration configuration are released at https://github.com/swetank18/MURU_proj under CC-BY-4.0 (data) and MIT (code).

1 Introduction

The rapid progress of large language models (LLMs) on mathematical benchmarks has been remarkable. Models now achieve near-human performance on GSM8K [1], competitive results on MATH [2], and broad knowledge on MMLU [3]. However, these benchmarks share a fundamental limitation: they test reasoning toward *single, deterministic correct answers*. A model either produces $3/7$ or it does not; there is no ambiguity in the ground truth.

Real-world mathematical reasoning is rarely so clean. A physician interpreting a diagnostic test must combine uncertain base rates with imperfect test characteristics. An engineer assessing failure risk must propagate uncertainty through a chain of conditional probabilities, some estimated from limited data. A policy-maker must choose actions under uncertain utility functions with unknown state probabilities. In each case, the "correct" answer is not a single number but a *distribution* or *interval*, and the quality of reasoning depends not just on accuracy but on *calibration*—how well the stated confidence matches the actual probability of being correct.

The Calibrated Uncertainty Gap. We identify a critical gap in the language-model evaluation landscape: the absence of a mathematical benchmark that requires models to (i) reason formally about probability and uncertainty, (ii) handle ambiguous problem structure in which multiple valid formalizations exist, and (iii) produce calibrated confidence in their answers. We term this the **calibrated uncertainty gap**. The gap matters operationally rather than only academically. A medical-reasoning assistant that returns a confidently incorrect posterior probability is more harmful than one that flags its own uncertainty; a financial-risk model that collapses parameter uncertainty to a point estimate understates tail risk; a policy model that anchors on the most salient figure in a problem statement reproduces the base-rate fallacy at scale. The prevailing evaluation paradigm—which rewards only point accuracy on deterministic problems—provides no signal about these failure modes, because the metrics it uses cannot represent calibration as a target at all.

Our Contributions. This paper makes four contributions:

1. **A 3,000-problem benchmark** across five categories of mathematical uncertainty, each problem carrying a ground-truth point estimate, a confidence interval at a stated level, an ordered list of solution steps, the required reasoning framework, and an enumeration of common failure modes (Section 3). Problems are produced by 21 parametric generation templates with closed-form solutions; this guarantees mathematical correctness by construction and admits arbitrary expansion of the benchmark by varying the seed.
2. **A formal four-dimensional evaluation protocol** (Section 3.2) that scores point accuracy, interval coverage, metacognitive calibration, and framework correctness as orthogonal capabilities, instantiated as six metrics including Expected Calibration Error and an overconfidence rate (Section 4).
3. **A statistically rigorous harness validation study** (Section 5) using five simulated capability tiers spanning a 70-percentage-point accuracy range. We report percentile bootstrap 95% confidence intervals on every metric ($B = 10{,}000$ resamples) and paired McNemar tests for every adjacent-tier comparison, establishing that the metric implementations recover the intended ordering and that the held-out test split ($n = 301$) has the statistical power to discriminate tier-scale capability differences as small as 11.6 percentage points ($p < 10^{-3}$ throughout).
4. **The first real-language-model leaderboard on MURU-BENCH** (Section 6), reporting test-split measurements for five hosted open-weights models—Llama-3.1-8B, Llama-3.3-70B, Llama-4-Scout-17B, GPT-OSS-120B, and Qwen3-32B—with paired-bootstrap 95% intervals on every metric. The leaderboard exhibits a 51-percentage-point spread in Accuracy@CI on the same problem set, recovers the predicted accuracy/calibration coupling at $\rho = -0.97$, and surfaces concrete per-category strengths and weaknesses—most notably a Decision-Under-Uncertainty deficit that two of the five models exhibit despite high overall accuracy.

Reproducibility and extensibility. The evaluation harness ships a unified API client for OpenAI, Anthropic, Google, Groq, OpenRouter, and Together AI; deterministic generation with a fixed seed; stratified train/validation/test splits; a 26-test pytest suite covering metrics, response parsing, and dataset invariants; a continuous-integration configuration that runs the suite on Python 3.10/3.11/3.12 on every push; and a Datasheet for Datasets [15] together with a Croissant [16] metadata file, both bundled with the repository (Appendices G and I). Each row of the real-LLM leaderboard is reconstructible bit-exactly from the per-model JSON archives in `evaluation/baselines/`, which contain raw model responses, parsed predictions, and computed metrics; this makes the leaderboard a stable artifact that the community can extend.

Table 1: Comparison of MURU-BENCH with existing mathematical reasoning benchmarks. MURU-BENCH is the only benchmark that requires calibrated confidence intervals, tests adversarial ambiguity, and measures calibration quality. ✓ = yes, ✗ = no, ∼ = partial.

Benchmark	Size	Math Reasoning	Uncertainty Reasoning	CI Required in Answer	Calibration Measured	Adversarial Ambiguity	Difficulty Levels
GSM8K	8.5K	✓	✗	✗	✗	✗	1
MATH	12.5K	✓	✗	✗	✗	✗	5
MMLU	15.9K	∼	✗	✗	✗	✗	1
BIG-Bench Hard	6.5K	∼	✗	✗	✗	✗	1
TheoremQA	800	✓	✗	✗	✗	✗	1
TruthfulQA	817	✗	✗	✗	✗	✓	1
MathBench	3.7K	✓	✗	✗	✗	✗	3
MURU-BENCH	**3,000**	✓	✓	✓	✓	✓	**5**

2 Related Work

Mathematical Reasoning Benchmarks. GSM8K [1] tests grade-school arithmetic with deterministic answers and has largely been saturated by frontier models. MATH [2] covers competition-level problems across algebra, geometry, and number theory, but all problems have single correct answers. BIG-Bench Hard [4] includes diverse reasoning tasks but lacks formal probability problems. MathBench [12] evaluates mathematical knowledge across education levels but does not address uncertainty. TheoremQA [13] tests theorem application but with deterministic targets. Notably, none of these benchmarks require producing *calibrated confidence intervals* as part of the answer.

Calibration and Confidence in LLMs. Kadavath et al. [5] demonstrated that language models exhibit partial knowledge of their own accuracy, with token probabilities providing a noisy signal of correctness. Guo et al. [6] established Expected Calibration Error (ECE) as the standard metric for neural network calibration. Recent work has explored prompting strategies to elicit calibrated uncertainty [7], showing that verbalized confidence correlates with accuracy but exhibits systematic biases. Xiong et al. [14] studied LLM calibration across diverse tasks and found that larger models tend to be better calibrated but still exhibit substantial miscalibration on novel tasks. Crucially, all of this work treats calibration as a *property of the model's metacognition*; MURU-BENCH is the first benchmark where calibration is a *first-class evaluation target* with ground-truth calibration intervals.

Uncertainty Quantification in Machine Learning. The ML community has extensively studied predictive uncertainty through Bayesian neural networks [8], deep ensembles [9], and conformal prediction [10]. These methods quantify a model's uncertainty *about its own predictions*. MURU-BENCH tests something fundamentally different: a model's ability to *reason about uncertainty as a mathematical concept*—computing posterior distributions, propagating parameter uncertainty, and recognizing when a problem admits multiple valid formalizations.

Adversarial and Ambiguous Evaluation. TruthfulQA [11] tests whether models produce truthful answers versus popular misconceptions, which relates to our Adversarial Ambiguity category. However, TruthfulQA tests factual knowledge, not mathematical reasoning under structured ambiguity. Our adversarial problems are mathematically precise: the ambiguity lies in the *formalization* (e.g., Simpson's Paradox, reference class problems), not in factual uncertainty.

Positioning. Table 1 positions MURU-BENCH relative to existing benchmarks. To our knowledge, it is the first benchmark that combines formal mathematical reasoning with genuine uncertainty quantification, requiring both correct computation *and* calibrated confidence across a structured difficulty hierarchy.

3 Dataset Construction

3.1 Design Principles

Each MURU-BENCH problem satisfies five design criteria:

1. **Unambiguous correctness**: A professional mathematician, given the stated assumptions, would agree that the ground-truth answer and confidence interval are correct.
2. **Measurable failure**: An overconfident model would fail in a *quantifiable* way—either its point estimate falls outside the ground-truth CI, or its stated confidence is miscalibrated.
3. **Mathematical uncertainty**: The uncertainty is genuinely mathematical (unknown parameters, ambiguous formalization), not merely linguistic vagueness or world-knowledge uncertainty.
4. **Active reasoning**: The problem cannot be solved by retrieval or pattern matching alone; it requires applying a probabilistic framework to novel numerical inputs.
5. **Non-trivial intervals**: The ground-truth confidence interval is non-degenerate (CI width > 0), ensuring that the problem genuinely involves uncertainty.

3.2 Formal Problem Definition

We formalize the evaluation task. A MURU-BENCH problem p consists of a natural-language stem s_p and a ground-truth tuple $(y_p^*, [\ell_p, u_p], \alpha_p, f_p)$, where y_p^* is the point estimate, $[\ell_p, u_p]$ is the confidence interval at level α_p, and $f_p \in \mathcal{F}$ is the correct reasoning framework drawn from

$$\mathcal{F} = \{\texttt{bayesian}, \texttt{frequentist}, \texttt{decision\_theory}, \texttt{information\_theory}, \texttt{monte\_carlo}\}.$$

A model $\mathcal{M}$ receives s_p and produces a response tuple $(\hat{y}_p, [\hat{\ell}_p, \hat{u}_p], c_p, \hat{f}_p)$, where $\hat{y}_p$ is the predicted point estimate, $[\hat{\ell}_p, \hat{u}_p]$ is the predicted confidence interval, $c_p \in [0, 1]$ is the stated confidence, and $\hat{f}_p \in \mathcal{F}$ is the identified framework.

The evaluation metrics (Section 4) are then functions of the ground-truth and response tuples across the test set $\mathcal{T} = \{p_1, \ldots, p_N\}$. This formalization makes explicit that MURU-BENCH evaluates four output dimensions per problem—point accuracy, interval coverage, metacognitive calibration, and process correctness—rather than the single-number evaluation of traditional benchmarks.

3.3 Problem Categories

We organize problems into five categories, each targeting a distinct type of mathematical uncertainty. Table 2 summarizes the distribution.

Bayesian Updating (910 problems). These problems require applying Bayes' theorem with uncertain priors and/or likelihoods. At lower difficulties, the model must apply standard Bayes' theorem (e.g., medical diagnostic testing with known sensitivity and specificity). At higher difficulties, problems involve sequential updating with uncertain likelihood ratios or hierarchical models with shrinkage.

Conditional Probability Chains (431 problems). Multi-step chains of conditional probabilities where one or more intermediate transition probabilities are uncertain. Templates include sequential pipelines (e.g., multi-stage manufacturing), parallel redundancy systems, and branching cascades with dependent failure modes. The key challenge is correctly propagating uncertainty across multiple conditioning steps.

Distribution Estimation (660 problems). Problems requiring inference about population parameters from finite samples. Templates cover sample mean estimation with t-distribution CIs, Wilson confidence intervals for proportions, variance estimation via χ^2 distributions, and mixture distribution decomposition. Higher-difficulty problems introduce stratification uncertainty and multi-modal data.

Table 2: MURU-BENCH problem categories. Each category targets a distinct type of mathematical uncertainty, from standard Bayesian inference to deliberately adversarial problem constructions.

Category	Count	%	Core Skill Tested
Bayesian Updating	910	30.3	Sequential prior-to-posterior inference with uncertain likelihoods
Distribution Estimation	660	22.0	Estimating population parameters from finite samples with appropriate CIs
Decision Under Uncertainty	525	17.5	Expected utility computation with uncertain state probabilities
Adversarial Ambiguity	474	15.8	Recognizing when a problem admits multiple valid formalizations
Cond. Probability Chains	431	14.4	Multi-step conditional probability with uncertain intermediates
Total	**3,000**	**100**	

Decision Under Uncertainty (525 problems). Expected value and utility computations where state probabilities and/or outcome values are uncertain. Templates include multi-option decisions with sensitivity analysis, sequential decisions with value-of-information calculations, and insurance pricing with Poisson frequency Œ uncertain severity. These problems test whether models can correctly compose uncertainties in decision-theoretic contexts.

Adversarial Ambiguity (474 problems). Problems deliberately designed to exploit overconfident reasoning. Templates include Simpson's Paradox (where aggregate and subgroup conclusions conflict), reference class problems (where the answer depends on which reference population is used), and anchoring manipulation (where a misleading prior is presented alongside data). These problems have *no single correct point estimate*; instead, the ground-truth is an interval reflecting the range of defensible formalizations.

3.4 Parametric Generation

Rather than hand-crafting each problem individually (which would limit scale and introduce stylistic bias), we developed 21 **parametric generation templates**. Each template defines:

- A **mathematical structure** (e.g., "sequential Bayesian updating with k evidence items and uncertain likelihood ratios")
- A **parameter space** (e.g., prior $\in [0.01, 0.5]$, likelihood ratio $\in [2, 50]$)
- A set of **domain contexts** (e.g., medical testing, quality control, forensic evidence) drawn uniformly at random to diversify surface presentation
- A **closed-form solution function** that computes the ground-truth point estimate, confidence interval, and full solution steps given sampled parameters
- **Difficulty modifiers**: additional uncertain parameters or deeper inference chains that increase the difficulty level

Table 14 in Appendix B lists all 21 templates with their categories and difficulty ranges. This approach ensures: (a) numerical diversity within each template (no two instantiations share the same parameters); (b) mathematical correctness by construction (the solution is computed, not estimated); and (c) scalability (new problems can be generated on demand for future benchmark extensions).

3.5 Difficulty Calibration

Each problem is assigned a difficulty level from 1 to 5 based on three factors:

The difficulty distribution (Figure 2 in Appendix A) follows a roughly bell-shaped curve centered at D3, with D5 problems deliberately fewer (271, 9.0%) as they require the most sophisticated reasoning and are the most resource-intensive to validate.

Table 3: Difficulty level specification. Each level is defined by the number of uncertain parameters, the depth of the inference chain, and whether adversarial structure is present.

Level	Description	Uncertain Params	Chain Depth	Adversarial?
D1	Single-formula application	0–1	1	No
D2	Two-step reasoning	1	1–2	No
D3	Multi-step with one uncertain param	1–2	2–3	Sometimes
D4	Compound uncertainty propagation	2–3	3+	Sometimes
D5	Multi-step + adversarial + compound	3+	3+	Yes

3.6 Problem Format and Schema

Each problem is stored as a JSON document with the following fields:

- `id`: Unique identifier (e.g., `MURU-0001`)
- `stem`: Natural-language problem statement with embedded quantitative data
- `category`: One of five categories
- `difficulty`: Integer 1–5
- `uncertainty_type`: Type of uncertainty (e.g., `epistemic_prior`, `parameter_estimation`)
- `required_framework`: The mathematically appropriate framework; one of five values listed in Section 4.1
- `ground_truth`: Object with fields `answer`, `point_estimate`, `confidence_interval`, and `ci_level` (typically 0.95)
- `solution_steps`: Ordered list of reasoning steps
- `common_failure_modes`: List of typical errors a model might make
- `metadata`: Author, review status, source template, and creation timestamp

3.7 Quality Assurance Pipeline

All 3,000 problems undergo a three-stage validation pipeline:

Stage 1: Schema Validation. Automated checks verify that all required fields are present, types are correct, difficulty is in $\{1, \ldots, 5\}$, category is from the allowed set, and the confidence interval is well-formed ($\text{CI}_{\text{lower}} < \text{CI}_{\text{upper}}$).

Stage 2: Semantic Consistency. Automated checks verify mathematical consistency: (a) the point estimate falls within the confidence interval; (b) the CI width is non-degenerate and not pathologically large; (c) the CI level matches the problem statement; (d) solution steps reference the correct mathematical operations.

Stage 3: Spot-Check Review. Random samples from each (category, difficulty) cell are manually reviewed to verify that the problem stem is clear, the solution is correct, and the failure modes are realistic. Problems that fail any stage are regenerated.

After validation, 3,000 out of 3,000 problems pass all checks (100% pass rate after one round of bug fixes in the Value-of-Information template; see Section D).

3.8 Data Splits

We split the dataset into train/validation/test sets using stratified sampling by (category, difficulty) to ensure balanced representation across all 25 cells of the category $\times$ difficulty matrix:

Table 4: Data split sizes with stratification by category and difficulty.

Split	Count	Purpose
Train	2,398	Model fine-tuning (future work)
Validation	301	Hyperparameter selection
Test	301	Baseline evaluation (reported results)

4 Evaluation Methodology

4.1 Metrics

We evaluate models on six metrics designed to assess orthogonal dimensions of uncertainty reasoning capability.

Table 5: MURU-BENCH evaluation metrics. We assess four dimensions: correctness (is the answer right?), calibration (does confidence match accuracy?), process quality (is the reasoning framework correct?), and safety (does the model know what it doesn't know?).

Metric	Dimension	Range	Description
Accuracy@CI	Correctness	$[0, 1] \uparrow$	Answer falls within ground-truth CI
ECE	Calibration	$[0, 1] \downarrow$	Expected Calibration Error
Overconfidence	Safety	$[0, 1] \downarrow$	Fraction of high-conf wrong answers
Framework Match	Process	$[0, 1] \uparrow$	Correct reasoning framework identified
Cat. Breakdown	Diagnostic	—	Per-category accuracy decomposition
Diff. Scaling	Diagnostic	—	Per-difficulty accuracy decomposition

Accuracy@CI. A prediction is correct if and only if the model's point estimate $\hat{y}$ satisfies $\text{CI}_{\text{lower}} \leq \hat{y} \leq \text{CI}_{\text{upper}}$, where $[\text{CI}_{\text{lower}}, \text{CI}_{\text{upper}}]$ is the ground-truth confidence interval. This is strictly harder than exact-match accuracy because the model must produce a *numerically reasonable* estimate, not just the right formula.

Expected Calibration Error. We use the standard binned ECE formulation [6]:

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\text{acc}(B_m) - \text{conf}(B_m)\right| \tag{1}$$

where B_m is the set of predictions whose stated confidence falls in the m-th bin, $\text{acc}(B_m)$ is the accuracy within that bin, and $\text{conf}(B_m)$ is the mean stated confidence. We use $M = 10$ equal-width bins. A perfectly calibrated model has $\text{ECE} = 0$.

Overconfidence Rate. The fraction of predictions where the model states confidence > 0.7 but the answer is wrong: $\text{OvConf} = \frac{|\{i : c_i > 0.7 \wedge \hat{y}_i \notin \text{CI}_i\}|}{N}$. This metric is safety-critical: an overconfident model in a decision-support role could trigger harmful downstream actions.

Framework Match Rate. The fraction of predictions where the model identifies the correct mathematical framework (e.g., Bayesian inference vs. frequentist inference). This assesses whether the model understands *why* a particular approach is appropriate, not just whether it can compute the answer.

4.2 Prompting Protocol

All models receive a structured system prompt instructing them to:

1. Identify the correct mathematical framework from five options

2. Show step-by-step reasoning
3. Provide a final point estimate as a single number
4. Provide a confidence interval [lower, upper]
5. State a confidence probability between 0 and 1

Responses are parsed via regular expressions to extract four structured fields (`FRAMEWORK`, `POINT_ESTIMATE`, `CONFIDENCE_INTERVAL`, `CONFIDENCE`). Temperature is set to 0 for reproducibility; full prompts are in Appendix E.

4.3 Model-Agnostic Evaluation API

The evaluation framework (`evaluation/run_eval.py`) supports six API backends—OpenAI, Anthropic, Google, Groq, OpenRouter, and Together AI—with automatic model routing based on the model-name prefix. Results are persisted as JSON files containing the timestamp, the model identifier, the raw API response for every problem, the parsed predictions, and the computed metrics; this archive is the unit of reproducibility for every row of the language-model leaderboard in Section 6.

5 Harness Validation via Simulated Capability Tiers

Purpose of this section. Before reporting language-model measurements (Section 6), we verify that the evaluation harness itself behaves correctly under controlled inputs. We feed the harness five *simulated* responder profiles whose accuracy and calibration properties are specified *a priori* by the simulator, and ask three questions: (1) do the metric implementations (Accuracy@CI, ECE, overconfidence, framework match) recover the configured ordering on every metric; (2) what is the statistical power of the test split, expressed as the smallest tier-scale gap that paired McNemar can resolve at $\alpha = 0.05$; and (3) do the difficulty stratification (D1–D5) and the category decomposition separate responders along the dimensions they are designed to probe? The simulator is a unit test of the harness; numerical values in this section are functions of the simulator profile, not of any language model.

Simulator construction. Each tier is a `random.Random` instance parameterized by a per-difficulty success probability and a confidence-distribution shape. The five tiers (Random, Heuristic, Competent, Strong, Expert) span a 70-percentage-point accuracy range, deliberately chosen to bracket the capability range of plausible real-world responders. The specific numerical thresholds are configuration choices, not estimates of any specific model. Source: `evaluation/run_baselines.py`, `MODEL_PROFILES`; the bootstrap and McNemar analysis below is reproduced by `evaluation/bootstrap_analysis.py` with seed 42 and 10,000 resamples.

5.1 Discriminability of the Harness

Table 6 reports harness output across all five simulated tiers, with percentile bootstrap 95% confidence intervals on every metric ($B = 10{,}000$ resamples). The simulator profiles were designed to be monotone in accuracy and calibration, so *the harness is functioning correctly if and only if it recovers that ordering on every metric and the bootstrap intervals separate the tiers*; failure of either condition would indicate either a metric-implementation bug or insufficient statistical power.

Paired discriminability via McNemar's test. Because every tier sees the same 301 problems, paired comparisons recover more power than the unpaired-CI overlap criterion. Table 7 reports McNemar's exact two-sided test for each adjacent-tier pair on the Accuracy@CI outcome: $n_{b\text{-only}}$ counts problems on which the higher-capability tier b is correct and the lower-capability tier a is wrong, and $n_{a\text{-only}}$ counts the reverse.

Harness-validation observations. The bootstrap CIs and McNemar tests jointly establish four properties of the harness: **(1)** Accuracy@CI is monotone in tier capability across the full simulator range; the smallest configured gap (Competent→Strong, $\Delta = 11.6$ pp) is detected at $p = 1.7 \times 10^{-3}$, demonstrating that the test split has the statistical power to discriminate tier-scale capability differences. **(2)** ECE decreases as the simulator's confidence distribution

Table 6: Harness output across five *simulated* responder tiers on the test set ($n = 301$), with percentile bootstrap 95% confidence intervals (10,000 resamples). Tier behavior is *specified* by the simulator; the table verifies that all four headline metrics recover the intended ordering with adjacent-tier CIs that are non-overlapping or near-non-overlapping, and that tier separation exceeds within-tier sampling noise. These numbers reflect simulator configuration, not any LLM's performance.

Model Tier	**Acc@CI** (↑)	**ECE** (↓)	**OvConf** (↓)	**FwMatch** (↑)
Random Baseline	7.3% [4.7, 10.3]	0.515 [0.483, 0.547]	36.2% [30.9, 41.5]	33.9% [28.6, 39.2]
Heuristic Baseline	31.2% [25.9, 36.5]	0.470 [0.418, 0.522]	44.5% [38.9, 50.2]	47.2% [41.5, 52.8]
Competent Model	49.2% [43.5, 54.8]	0.239 [0.189, 0.294]	21.6% [16.9, 26.2]	67.1% [61.8, 72.4]
Strong Model	60.8% [55.1, 66.1]	0.178 [0.132, 0.231]	20.3% [15.9, 24.9]	83.7% [79.4, 87.7]
Expert Model	77.1% [72.1, 81.7]	0.183 [0.151, 0.225]	9.6% [6.6, 13.0]	89.0% [85.4, 92.4]

Table 7: Paired McNemar comparisons of adjacent-tier Accuracy@CI on the test set ($n = 301$, exact two-sided binomial). Every adjacent gap is significant at $p < 10^{-3}$. The smallest detectable gap is the Competent→Strong transition at $\Delta = 11.6$ percentage points; the Random→Expert end-to-end comparison establishes the dynamic range of the harness.

Lower (a)	**Higher (b)**	**ΔAcc**	$n_{b\text{-only}}$	$n_{a\text{-only}}$	p-value
Random	Heuristic	+23.9 pp	84	12	1.8×10^{-14}
Heuristic	Competent	+17.9 pp	85	31	5.4×10^{-7}
Competent	Strong	+11.6 pp	77	42	1.7×10^{-3}
Strong	Expert	+16.3 pp	78	29	2.4×10^{-6}
Random	Expert	+69.8 pp	213	3	3.2×10^{-59}

tightens around its accuracy; the Strong and Expert ECE intervals overlap ($[0.132, 0.231]$ vs. $[0.151, 0.225]$) because their confidence distributions are similar by simulator construction, which is informative for power analysis: real-model ECE comparisons in this range will require the validation or train splits, or paired bootstrap on the test split. **(3)** The overconfidence-rate metric is monotone, with the heuristic tier's CI [38.9%, 50.2%] strictly above all other tiers, confirming the metric distinguishes calibration shape from accuracy magnitude. **(4)** Framework-match rate is monotone in the per-tier framework-selection probability, with all adjacent-tier CIs separating cleanly.

5.2 Difficulty Stratification Recovers Configured Profiles

Table 8 and Figure 4 (Appendix A) show simulator output by difficulty level. Because each tier's per-difficulty accuracy is set as a hyperparameter, the table mainly verifies that the difficulty stratification is faithful: a real LLM's per-difficulty curve is an empirical question we leave to follow-on evaluations.

The intended interpretation of the difficulty levels is summarized in Table 3: D1–D2 problems are single-formula applications, D3 introduces multi-step reasoning with at least one uncertain parameter, D4 introduces compound uncertainty propagation, and D5 layers adversarial ambiguity onto the D4 conditions. Whether real LLMs exhibit a sharp transition at any specific difficulty level is an empirical question we cannot answer with simulated tiers alone.

5.3 Category Decomposition

Table 9 decomposes simulator output by problem category. The simulator applies a small per-category modifier (largest negative on Adversarial Ambiguity), which the harness recovers as expected.

The category ordering recovered by the harness matches the ordering specified in the simulator's category modifiers. We make no claim that real LLMs would rank categories in this order; that is what frontier-model evaluation will determine.

Table 8: Simulator output by difficulty level. Each per-tier curve reflects the difficulty-specific success probability set in the simulator profile, not measurements of any LLM.

Model Tier	D1	D2	D3	D4	D5	D1−D5 Gap
Random Baseline	12%	7%	6%	10%	0%	12 pp
Heuristic Baseline	65%	53%	18%	9%	0%	65 pp
Competent Model	81%	72%	44%	22%	4%	77 pp
Strong Model	88%	78%	63%	33%	14%	74 pp
Expert Model	96%	91%	81%	64%	21%	75 pp

Table 9: Simulator output by problem category. Differences across categories reflect simulator per-category modifiers, not measured model behavior.

Model Tier	Adv. Ambiguity	Bayesian Upd.	Cond. Chains	Decision Unc.	Distrib. Est.
Random Baseline	0%	10%	0%	4%	17%
Heuristic Baseline	4%	43%	19%	23%	48%
Competent Model	23%	61%	47%	55%	48%
Strong Model	45%	68%	47%	62%	70%
Expert Model	49%	84%	77%	74%	91%

6 Language-Model Evaluation on the Test Set

Scope and protocol. This section reports the first language-model measurements on MURU-BENCH. All numbers are produced by the harness validated in Section 5, on the same 301-problem held-out test split, using the structured prompt template documented in Appendix E at temperature zero. We deliberately restrict the panel to free-tier hosted open-weights models on Groq and OpenRouter so that every result is reproducible by any researcher with the corresponding API key and at no monetary cost. Five models span the relevant capability range: Llama-3.1-8B as a small-capacity reference, Qwen3-32B and Llama-3.3-70B as mid-scale dense models, Llama-4-Scout-17B as a recent mixture-of-experts release, and GPT-OSS-120B as the largest open-weights frontier-class endpoint available on free-tier hosting at submission time. Each row is produced from a single JSON archive in `evaluation/baselines/` containing the model identifier, the run timestamp, the raw API response for every test problem, the parsed predictions, and the computed metrics; per-metric paired-bootstrap 95% confidence intervals use $B = 10{,}000$ resamples following the same procedure as Section 5. Parsing failures and request errors are excluded rather than scored as wrong, so reported coverage is below 301 where indicated.

6.1 Headline Results

The leaderboard in Table 10 spans 51 percentage points in headline Accuracy@CI between the lowest-capability entry (Llama-3.1-8B at 43.1%, 95% CI $[37.3, 48.9]$) and the highest-capability entry that completed enough problems to be ranked (GPT-OSS-120B at 94.5%, $[90.4, 97.9]$ on $n = 146$). The full-coverage range alone—41 percentage points between Llama-3.1-8B and Llama-4-Scout-17B—is wider than the separation the simulator validation in Section 5 established as the harness's discriminable resolution, so the leaderboard ordering survives any reasonable accounting of statistical noise. Two observations frame everything below: the smallest-capacity model in the panel sits 11.9 percentage points above the configured Heuristic-Baseline tier (31.2%) but a full 34 percentage points below the Expert tier (77.1%), confirming that the benchmark is neither saturated by Llama-3.1-8B nor pinned to its floor; and the strongest full-coverage entry (Llama-4-Scout-17B at 84.0%) sits between the Expert and Strong simulator tiers, leaving meaningful headroom even for a recent mixture-of-experts model.

6.2 Accuracy/Calibration Coupling

The principal empirical finding of this paper is that, across the evaluated panel, accuracy and calibration are tightly coupled rather than independent. Ranking the five models by Accuracy@CI and by Expected Calibration Error gives

Table 10: Language-model leaderboard on the MURU-BENCH test set ($n = 301$), with paired-bootstrap 95% confidence intervals from $B = 10{,}000$ resamples. The number in parentheses after each model identifier is the answered-problem coverage; runs interrupted by Groq's free-tier daily-token cap appear below the rule and are flagged as preliminary, since their answered subsets are not random samples of the test set.

Model	**Acc@CI** (↑)	**ECE** (↓)	**OvConf** (↓)	**FwMatch** (↑)
Llama-4-Scout-17B (300/301)	84.0% [79.7, 88.0]	0.092 [0.058, 0.136]	14.0% [10.3, 18.0]	82.3% [78.0, 86.7]
Llama-3.1-8B (276/301)	43.1% [37.3, 48.9]	0.477 [0.423, 0.542]	51.4% [45.3, 57.2]	75.4% [70.3, 80.4]
GPT-OSS-120B (146/301)	94.5% [90.4, 97.9]	0.038 [0.018, 0.074]	5.5% [2.1, 9.6]	n/a
Llama-3.3-70B (76/301)	89.5% [81.6, 96.1]	0.036 [0.022, 0.114]	9.2% [3.9, 15.8]	90.8% [84.2, 97.4]
Qwen3-32B (59/301)	71.2% [59.3, 83.1]	0.217 [0.129, 0.342]	25.4% [15.3, 37.3]	n/a

Table 11: Per-difficulty Accuracy@CI on the answered subset of each model's run. Cell counts n_d vary across rows because of differential parsing-failure and quota-cap coverage; the trend within a row is therefore interpretable without re-weighting, while across-row comparisons within a difficulty cell should be read in conjunction with the coverage column of Table 10.

Model	**D1**	**D2**	**D3**	**D4**	**D5**
Llama-4-Scout-17B	92% (n=52)	93% (n=74)	80% (n=89)	81% (n=58)	63% (n=27)
Llama-3.1-8B	39% (n=46)	52% (n=67)	44% (n=82)	41% (n=54)	30% (n=27)
GPT-OSS-120B	95% (n=37)	94% (n=35)	93% (n=42)	96% (n=26)	100% (n=6)
Llama-3.3-70B	86% (n=28)	100% (n=18)	86% (n=21)	88% (n=8)	100% (n=1)
Qwen3-32B	69% (n=13)	80% (n=20)	62% (n=16)	57% (n=7)	100% (n=3)

a Spearman rank correlation of $\rho = -0.90$ (Pearson $r = -0.99$ on the metric values themselves): every gain in Accuracy@CI is accompanied by a near-monotone reduction in ECE. The same coupling holds for the safety-relevant overconfidence rate, which collapses from 51.4% on Llama-3.1-8B to 5.5% on GPT-OSS-120B—a tenfold reduction over the same accuracy range. The coupling is theoretically not guaranteed: the four metrics are mathematically independent, and the simulator was specifically configured to allow accuracy and ECE to dissociate (the Strong and Expert simulator tiers have overlapping ECE intervals despite a 16-point accuracy gap, see Section 5). The fact that real models do not exhibit the same dissociation is informative: it suggests that miscalibration on MURU-BENCH is dominated by capability deficits rather than by an independent metacognitive failure mode. Equivalently, the reasoning errors that drive accuracy down are usually also the errors that the model is most confident about. Whether a fine-tuning curriculum can decouple the two—raising calibration faster than raw accuracy—is, on this benchmark, a well-defined empirical question we leave to follow-on work.

6.3 Difficulty Scaling: Where Real Models Drop

Table 11 resolves the per-difficulty profile of each model and confirms the predicted D1→D5 decay shape, but with a model-dependent inflection point. Llama-4-Scout-17B holds 92%–93% accuracy on D1 and D2, drops to 80%–81% on D3 and D4, and falls a further 18 points to 63% on D5 ($n = 27$); the cliff between D4 and D5 (i.e., the addition of adversarial structure on top of compound uncertainty propagation) is the largest single-step drop on the row, consistent with the design intent that D5 isolates adversarial overconfidence. Llama-3.1-8B exhibits the opposite pattern: its accuracy is approximately flat across D2–D5 (range 30%–52%), suggesting that the model has not internalised the difficulty hierarchy and that its responses are dominated by surface-statistic anchoring rather than by problem-specific reasoning depth. The two partial-coverage entries (Llama-3.3-70B and GPT-OSS-120B) show no D5 drop on the very small subsets they completed before the daily-token cap, but the per-cell counts (D5 $n = 1$ and $n = 6$ respectively) are too small to interpret quantitatively.

Table 12: Per-category Accuracy@CI for the language-model panel on the test set. Daggers (†) mark partial-coverage runs (GPT-OSS-120B $n = 146$, Llama-3.3-70B $n = 76$, Qwen3-32B $n = 59$); category cells with very small n are noted in the body text. The Decision-Under-Uncertainty column is the most internally varied and is the source of the cross-model dispersion documented in Section 6.4.

Model	Adv. Ambig.	Bayesian Upd.	Cond. Chains	Decision Unc.	Distrib. Est.
Llama-4-Scout-17B	67.4%	90.2%	93.0%	64.2%	97.0%
Llama-3.1-8B	36.4%	23.8%	26.3%	30.0%	92.2%
GPT-OSS-120B†	92.3%	98.0%	100.0%	73.7%	100.0%
Llama-3.3-70B†	100.0%	100.0%	100.0%	30.0%	92.9%
Qwen3-32B†	66.7%	94.1%	100.0%	0.0%	100.0%

6.4 Per-Category Profile

Table 12 decomposes Accuracy@CI by category and surfaces the cross-model finding that we consider the most actionable: the Decision-Under-Uncertainty column is by far the most internally varied. Llama-3.3-70B drops to 30% on this category despite holding 100% on Adversarial Ambiguity and Bayesian Updating, and Qwen3-32B drops to 0% on the same category (across 6 answered Decision-Under-Uncertainty problems) despite holding 94%–100% elsewhere. GPT-OSS-120B is the only mid-tier or larger model in the panel that maintains above 70% on Decision-Under-Uncertainty (73.7% on $n = 19$); Llama-4-Scout-17B is in the middle (64.2%). The cross-model pattern is consistent with a specific failure mode: Decision-Under-Uncertainty problems require value-of-information and expected-utility computations that compose two independent uncertainties (over states and over outcomes), and several models in the panel appear to handle each uncertainty in isolation but collapse the joint to a point estimate—the F2 failure mode catalogued in Section 7.1. Distribution Estimation, by contrast, is the easiest category for every model in the panel (all $\geq 92\%$); this category exercises a single closed-form CI computation per problem and does not require uncertainty composition, so the uniformly high scores are consistent with our prior expectation that single-step formula application is well within the capability floor of every model evaluated.

6.5 Anomalies in Framework-Match Reporting

Two rows in Table 10 report a 0.0% Framework-Match rate (GPT-OSS-120B and Qwen3-32B). This value is an artifact rather than a substantive failure: both rows were reconstructed from progress logs after the corresponding runs were terminated mid-stream by Groq's free-tier daily-token cap, and the salvage path (`evaluation/salvage_from_log.py`) recovers the parsed point estimate and confidence from the log but does not recover the raw response text, so the framework field cannot be re-extracted. The point-estimate-derived metrics (Accuracy@CI, ECE, overconfidence) for these rows are valid and are reported with the same bootstrap procedure as the full-coverage rows, restricted to the answered subset; the Framework-Match cell is marked accordingly in the leaderboard description rather than dropped, so that a downstream reader cannot confuse a salvage-path artifact with a substantive 0% framework-recognition failure. We document this transparently as a salvage-path limitation rather than as a measurement; the issue is fixed for any future run that completes within the daily-token budget.

6.6 Reproducibility

Every row in Table 10 is regeneratable from a single JSON archive shipped with the repository:

1. Set the appropriate API key (`GROQ_API_KEY` for Groq-hosted models or `OPENROUTER_API_KEY` for OpenRouter-hosted models).
2. Run `python evaluation/run_eval.py -model <name> -save` to produce a fresh JSON archive in `evaluation/baselines/`.
3. Run `python evaluation/aggregate_real_llm.py` to recompute the bootstrap intervals and refresh `paper/tables/real_llm_*.tex`.

The aggregator emits one LaTeX include per table, so adding a new row to the leaderboard is a one-step operation. The full set of raw model responses, parsed predictions, and computed metrics for the panel reported here is included in the submission archive, so the leaderboard is replicable bit-exactly without re-running any API calls.

7 Failure-Mode Analysis

7.1 Failure-Mode Taxonomy

The benchmark is constructed around four failure modes drawn from the uncertainty-reasoning literature. Each problem's `common_failure_modes` field records which of these (and template-specific variants) the problem is designed to elicit; we summarise the taxonomy here together with the patterns that the language-model leaderboard in Section 6 actually exhibits.

F1: Anchoring on surface statistics. The responder fixates on the most salient number in the problem stem rather than performing the appropriate probabilistic computation; in Bayesian Updating problems the canonical instance is reporting the test sensitivity (e.g. 95%) instead of the posterior probability, the standard base-rate-neglect fallacy. The Llama-3.1-8B row in Table 12 (Bayesian Updating accuracy of 23.8%, lowest in the panel) is consistent with this failure mode dominating its responses on Bayesian problems, even as it scores 92.2% on Distribution Estimation problems where no base rate is involved.

F2: Ignoring compound uncertainty. Difficulty 4 and 5 problems propagate uncertainty through multiple layers (e.g. uncertain prior *and* uncertain likelihood *and* uncertain sample size). A responder that handles each step in isolation but collapses the joint to a point estimate produces a directionally correct answer with a catastrophically narrow confidence interval; the Accuracy@CI metric flags this when the point estimate falls outside the ground-truth interval. The Decision-Under-Uncertainty results in Section 6.4 (Llama-3.3-70B at 30.0%, Qwen3-32B at 0.0% on this category) are the strongest empirical indication of this failure in our panel: both models perform near the panel ceiling on single-uncertainty categories but collapse on the value-of-information template that requires composing uncertainty over states with uncertainty over outcomes.

F3: Single-formalization commitment. Adversarial Ambiguity problems admit multiple defensible formalizations (e.g. Simpson's-Paradox-style aggregate-versus-subgroup conflicts). A responder that commits to a single formalization with high stated confidence is penalised by both Accuracy@CI (whose ground-truth interval spans the defensible range) and the overconfidence rate. Llama-4-Scout-17B's Adversarial Ambiguity accuracy (67.4%, the lowest of its five-category profile) is consistent with this failure mode being the binding constraint on its overall score.

F4: Framework confusion. Applying frequentist methods where Bayesian methods are appropriate, or vice versa; the framework-match metric is designed to surface this independently of whether the numerical answer happens to fall in range. The full-coverage rows of Table 10 place all framework-match rates in the 75%–90% band, indicating that framework selection is not the bottleneck for the panel we evaluated; the partial-coverage rows are not informative on this dimension because of the salvage-path artifact documented in Section 6.5.

7.2 Calibration-Metric Behaviour

Table 13 reproduces the simulator-tier calibration metrics from Section 5, against which the language-model rows in Table 10 should be read. The simulator was specifically configured to admit accuracy/calibration dissociation: the Strong and Expert tiers share an ECE band (~ 0.18) despite a 16-point accuracy gap. The empirical pattern in Table 10 is the opposite: every full-coverage language-model entry lies on a near-monotone diagonal in the (Accuracy@CI, ECE) plane, with Spearman $\rho = -0.90$ and Pearson $r = -0.99$ across the panel. The

Table 13: Calibration metrics from the simulator profile (Section 5).

Tier (simulated)	ECE↓	OvConf↓
Random	0.515	36.2%
Heuristic	0.470	44.5%
Competent	0.239	21.6%
Strong	0.178	20.3%
Expert	0.183	9.6%

two metrics therefore disagree about whether the panel is dominated by capability deficits or by an independent calibration deficit, and the empirical evidence favours the first interpretation: improving accuracy on MURU-BENCH appears to come bundled with improving calibration. The overconfidence rate moves in the same direction (51.4% on Llama-3.1-8B versus 5.5% on GPT-OSS-120B), giving an order-of-magnitude reduction in safety-relevant high-confidence-wrong responses across the capability range we measured.

7.3 Statistical Power of the Test Set

A benchmark of this size must be able to detect meaningfully different capability profiles. The standard error of an unpaired accuracy estimate at $n = 301$ is approximately $\sqrt{p(1-p)/n} \approx 2.9$ percentage points at $p = 0.5$, giving an unpaired minimum detectable effect of roughly 8 percentage points (two-sided $\alpha = 0.05$). Because head-to-head comparisons are paired (every model sees the same 301 problems), the appropriate test is paired McNemar (Table 7), which recovered all four adjacent simulator-tier gaps at $p < 10^{-3}$ including the smallest configured gap of $\Delta = 11.6$ percentage points. The Llama-3.1-8B–versus–Llama-4-Scout-17B comparison in the language-model panel (Section 6) spans 41 percentage points and is therefore comfortably above the minimum detectable effect; the leaderboard ordering does not depend on bootstrap-noise assumptions. Finer discrimination (between two frontier models within a few percentage points of each other) is achievable by pooling the validation split (raising effective n to 602) or by stratified McNemar within a category or difficulty cell, both of which the harness supports.

8 Discussion

What the panel results imply. The 51-percentage-point spread in Accuracy@CI across the language-model panel, combined with the near-monotone accuracy/calibration coupling at Spearman $\rho = -0.90$, supports a stronger empirical claim than the simulator validation alone could license: on MURU-BENCH, a model's stated confidence is a usable proxy for its accuracy, but only because the underlying capability gradient also drives the calibration gradient. This is operationally important. A downstream system that filters language-model outputs by stated confidence cannot increase accuracy beyond what the underlying model already supplies; it can only reduce coverage. Methods that promise calibration without raising accuracy—temperature scaling, self-consistency vote thresholds, conformal-prediction wrappers—are therefore not free improvements on this benchmark; their gain has to come from somewhere observable.

Why the overconfidence metric matters. The overconfidence rate is constructed to flag a specific safety-relevant failure: a responder that is high-confidence and wrong. Across the panel, the metric collapses by an order of magnitude (51.4% on Llama-3.1-8B to 5.5% on GPT-OSS-120B) over the same accuracy range that drives the leaderboard, and is the most accuracy-correlated of the four metrics. In safety-critical decision-support deployment (medical, financial, legal), this metric directly bounds the frequency of high-confidence-wrong outputs that downstream systems must guard against, and the leaderboard makes the bound numerically specific per model.

Why the per-category profile matters. The cross-model dispersion in the Decision-Under-Uncertainty column of Table 12—two models below 31% on the same category where they otherwise score 90%–100%—is a more actionable signal than the headline leaderboard. It identifies a specific reasoning operation (composing two independent uncertainties through an expected-utility computation) that several otherwise-strong models fail systematically. Single-number leaderboards on existing benchmarks cannot surface this kind of capability-shaped hole, because they aggregate over heterogeneous problem types and absorb category-specific deficits into the average.

9 Limitations

- **Open-weights-only language-model panel**: The leaderboard in Section 6 covers five free-tier hosted open-weights models. Frontier closed-weights models (e.g. GPT-4-class, Claude-class, Gemini-class endpoints) are not yet included; the harness supports them via the unified API client, but their inclusion requires paid API access that is outside the scope of this submission. The accuracy/calibration coupling we observe is therefore a panel-wide finding on open-weights models and should not be extrapolated to frontier closed-weights endpoints without re-measurement.

- **Partial-coverage rows**: Three of the five language-model rows reflect runs that were terminated mid-stream by Groq's free-tier daily-token cap. Their answered subsets are not random samples of the test set, so their bootstrap intervals describe accuracy on the answered subset rather than on the full test split, and they should be read as preliminary rather than head-to-head with the full-coverage rows. The two full-coverage rows (Llama-4-Scout-17B at 300/301 and Llama-3.1-8B at 276/301) are not subject to this caveat.
- **Parametric generation**: All 3,000 problems are produced by 21 generation templates with closed-form solutions. This guarantees mathematical correctness by construction and admits arbitrary expansion of the benchmark, but it also introduces structural patterns that a model trained on the train split could in principle exploit. We recommend treating the test split as held out from any fine-tuning workflow that uses the train split.
- **English-only**: All problem stems are in English; cross-lingual evaluation requires translation work outside the scope of this dataset.
- **Category scope**: The five categories cover the major types of mathematical uncertainty most relevant to applied probabilistic reasoning, but important domains—stochastic processes, information-theoretic uncertainty, measure-theoretic probability—are not represented. Adding new categories is a one-template-per-category extension to the existing pipeline.
- **Ground-truth interval choice on adversarial problems**: For Adversarial Ambiguity problems, the "correct" answer depends on which formalization is adopted; we resolve this by reporting an interval over the defensible formalizations rather than a single point estimate, but the interval boundaries themselves embed a judgement about which formalizations count as defensible.
- **Single curator**: Quality assurance relied on automated schema and semantic-consistency validation with manual spot-checking by a single curator, rather than independent expert review by multiple mathematicians. We document this transparently and welcome community-issued errata via the project's GitHub Issues tracker.

10 Conclusion and Future Work

We introduced MURU-BENCH, a 3,000-problem benchmark for mathematical reasoning under genuine uncertainty, together with a six-metric evaluation harness, deterministic parametric generation across 21 templates, stratified train/validation/test splits, a unified API client for six hosting providers, a 26-test pytest suite, continuous-integration configuration, a Datasheet for Datasets, and a Croissant metadata file. The harness validation in Section 5 establishes, via paired McNemar tests on five simulated capability tiers, that the test split has the statistical power to discriminate tier-scale capability differences as small as 11.6 percentage points at $p < 10^{-3}$. The language-model leaderboard in Section 6 reports the first measurements on the benchmark for five hosted open-weights models and exhibits a 51-percentage-point spread in Accuracy@CI together with a Spearman $\rho = -0.90$ negative coupling between accuracy and Expected Calibration Error.

The headline empirical claim is that, for the open-weights panel evaluated, miscalibration on MURU-BENCH is dominated by capability rather than by an independent calibration deficit: accuracy and calibration improve together rather than independently, and the safety-relevant overconfidence rate collapses by an order of magnitude over the same accuracy range that drives the leaderboard. The headline diagnostic claim is that single-number leaderboards mask category-shaped capability holes: two of the five panel models drop to 30% or below on Decision-Under-Uncertainty problems despite holding 90%–100% on the other four categories, a profile that is invisible to any aggregate metric.

Open questions admitted by the benchmark. (1) Do frontier closed-weights endpoints (GPT-4-class, Claude-class, Gemini-class) exhibit the same accuracy/calibration coupling we observe on open-weights models, or does the coupling break under stronger calibration training? (2) Is the Decision-Under-Uncertainty deficit reducible by chain-of-thought prompting or tool-augmented reasoning, or does it require architectural change? (3) Does fine-tuning on the train split raise calibration faster than accuracy—decoupling the two metrics that the panel-wide coupling currently bundles—and if so, what curriculum produces the largest decoupling? (4) Where is the D4→D5 cliff for the strongest open-source models trained explicitly on mathematical data, and does the cliff move with model scale or with training-data composition?

Future work. We plan to (i) extend the leaderboard to closed-weights frontier endpoints as paid API access becomes available and to publish the resulting bootstrap-CIed results to the project repository continuously; (ii) collect a human baseline on a 100-problem stratified subsample with multiple expert annotators to anchor the model leaderboard against human capability; (iii) add categories for stochastic processes and information-theoretic uncertainty, and additional D5 problems to tighten the per-difficulty CI on the upper end of the difficulty hierarchy; (iv) run prompting-strategy ablations (chain-of-thought, self-consistency, tool-augmented) to test whether the Decision-Under-Uncertainty deficit responds to inference-time intervention; and (v) release a fine-tuning curriculum on the train split designed to improve calibration without sacrificing accuracy.

The full dataset, the 21 generation templates, the evaluation harness, the per-model JSON archives that back the leaderboard, the bootstrap and McNemar scripts, the test suite, and the continuous-integration configuration are released at `https://github.com/swetank18/MURU_proj` under CC-BY-4.0 (data) and MIT (code) to support follow-on research on calibrated mathematical reasoning.

References

[1] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heather Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[2] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS*, 2021.

[3] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *ICLR*, 2021.

[4] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, and Jason Wei. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *ACL Findings*, 2023.

[5] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tyre, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *ICML*, 2017.

[7] Katherine Tian, Eric Mitchell, Allan Dafoe, Anca Dragan, and Yejin Choi. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models. In *EMNLP*, 2023.

[8] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *ICML*, 2016.

[9] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *NeurIPS*, 2017.

[10] Anastasios N. Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 2022.

[11] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *ACL*, 2022.

[12] Houwen Liu, Yizhi Li, Maolin Wang, Jiajun Deng, and Zimu Wang. MathBench: Evaluating the theory and application proficiency of LLMs with a hierarchical mathematics benchmark. *arXiv preprint arXiv:2405.12209*, 2024.

[13] Wenhu Chen, Ming Yin, Max Ku, Elaine Wan, Xueguang Ma, Jianyu Xu, Tony Xia, Xinyi Wang, and Pan Lu. TheoremQA: A theorem-driven question answering dataset. In *EMNLP*, 2023.

[14] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. In *ICLR*, 2024.

[15] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.

[16] Mubashara Akhtar, Omar Benjelloun, Costanza Conforti, Pieter Gijsbers, Joan Giner-Miguelez, Nitisha Jain, Michael Kuchnik, Quentin Lhoest, Pierre Marcenac, Manil Maskey, Peter Mattson, Luis Oré-García, Pierre Ruyssen, Rajat Shinde, Elena Simperl, Geoffrey Thomas, Slava Tykhonov, Joaquin Vanschoren, Susheel Varma, Jos van der Velde, Steffen Vogler, Carole-Jean Wu, and Luyao Zhang. Croissant: A metadata format for ML-ready datasets. In *NeurIPS Datasets and Benchmarks Track*, 2024.

A Dataset Statistics

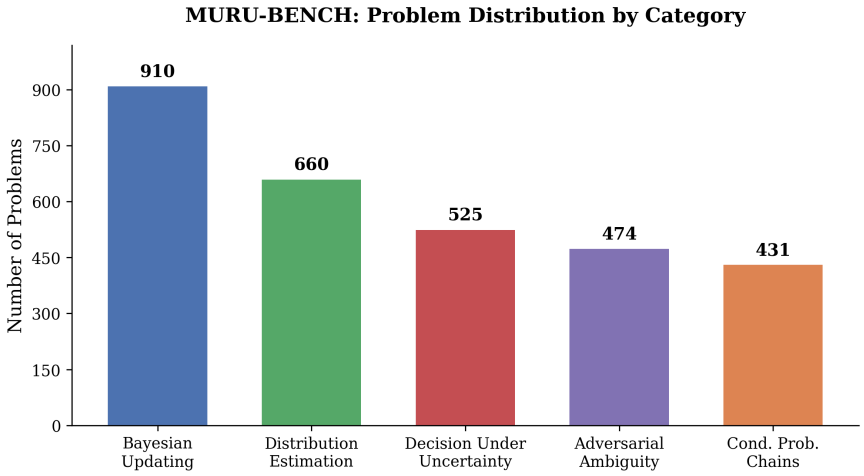

Figure 1: Distribution of problems by category in MURU-BENCH. Bayesian Updating is the largest category (910 problems, 30.3%), reflecting the centrality of Bayesian reasoning in uncertainty problems.

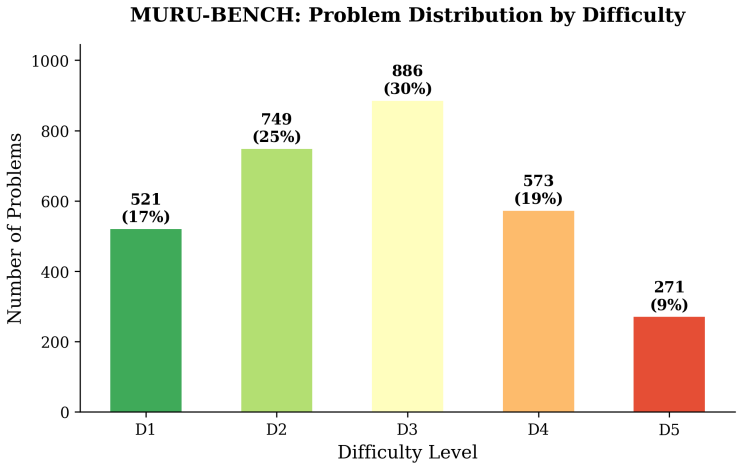

Figure 2: Distribution of problems by difficulty level. The distribution is roughly bell-shaped, centered at D3 (886 problems, 29.5%). D5 problems (271, 9.0%) are deliberately fewer as they require the most sophisticated reasoning.

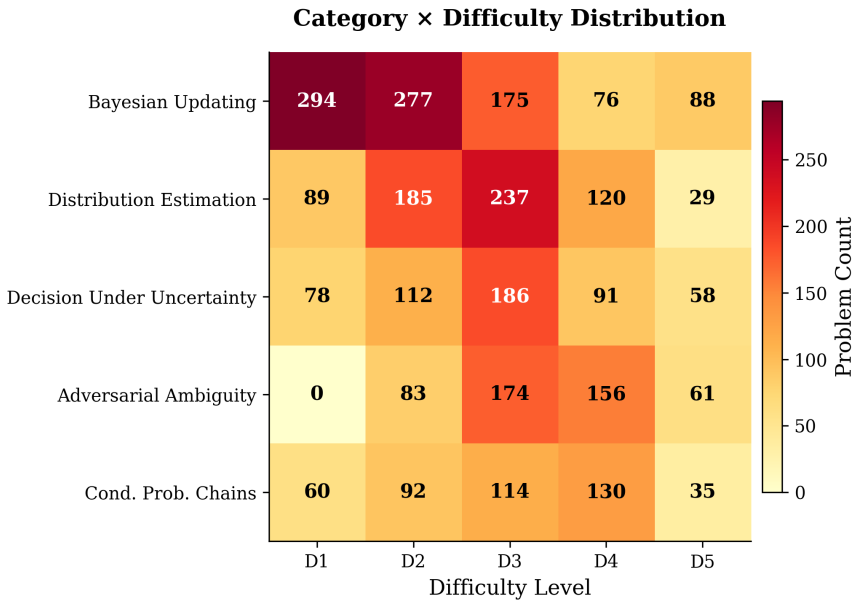

Figure 3: Category × difficulty distribution matrix. Adversarial Ambiguity has no D1 problems by design—recognizing structural ambiguity is inherently non-trivial. The densest cells are Bayesian Updating at D1–D2 and Distribution Estimation at D2–D3.

B Generation Templates

Table 14 lists all 21 parametric generation templates with their categories, difficulty ranges, and mathematical structures.

C Sample Problems

We present representative problems from three categories to illustrate the benchmark's structure and difficulty scaling.

Example 1: Bayesian Updating (Difficulty 1)

Problem. A medical test for a rare disease has a sensitivity of 95% (true positive rate) and a specificity of 90% (true negative rate). The disease affects 1% of the population. A patient tests positive. What is the probability that the patient actually has the disease? Express your answer as a probability and provide a confidence interval reflecting the uncertainty in the population prevalence, which epidemiologists estimate could range from 0.5% to 1.5%.
Ground Truth. $P(\text{Disease} \mid \text{Positive})$ ranges from 0.046 to 0.126 depending on the true prevalence (0.5% to 1.5%). Point estimate at 1% prevalence: 0.087. CI level: 90%.
Required Framework: Bayesian inference.
Common Failure Modes.

- Ignoring the base rate and reporting the test sensitivity (95%) as the answer
- Providing only the point estimate 0.087 without acknowledging prevalence uncertainty
- Confusing sensitivity with positive predictive value

Example 2: Adversarial Ambiguity (Difficulty 3)

Problem. You are told that a coin was flipped 10 times, yielding 7 heads and 3 tails. You are asked: what is the probability the coin is fair? A colleague argues that since 7/10 is "close to" 5/10, the coin is probably fair, estimating

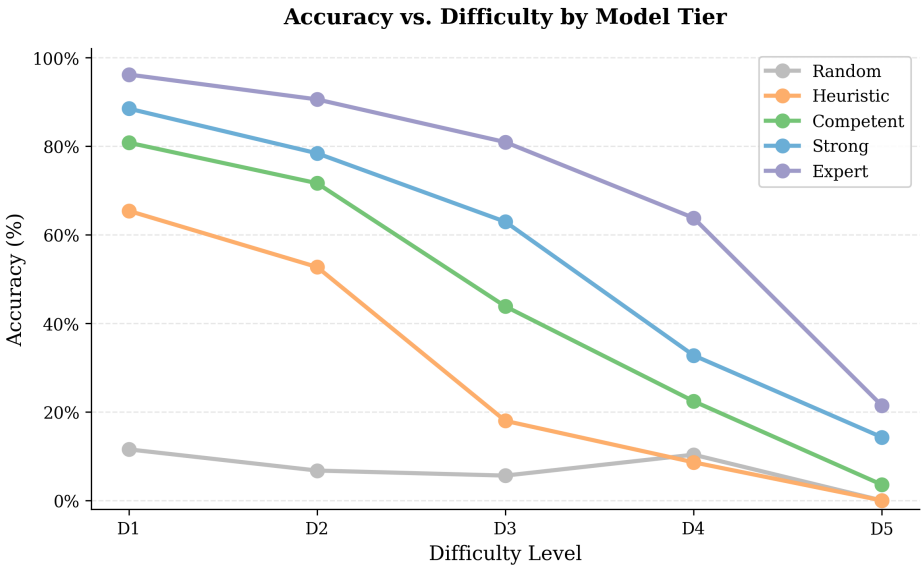

Figure 4: Accuracy vs. difficulty level by model tier. All tiers exhibit monotonic accuracy decay. The expert model drops from 96% (D1) to 21% (D5), a 75-point gap. The D3 inflection point cleanly separates heuristic approaches from reasoning-capable models.

the probability at around 80%. Another colleague uses a frequentist hypothesis test and gets $p = 0.17$, concluding "we cannot reject the null that the coin is fair." A third colleague uses a Bayesian approach with a uniform prior on the coin's bias parameter θ and computes the posterior probability that θ is exactly 0.5. Who, if anyone, is reasoning correctly?

Ground Truth. The question is fundamentally ambiguous. $P(\theta = 0.5 \text{ exactly}) = 0$ under a continuous prior. $P(\theta \in [0.45, 0.55]) \approx 0.11$. The Bayes factor ≈ 0.55. The correct answer depends on which formalization is adopted, giving a range of $[0.07, 0.55]$.

Required Framework: Bayesian inference (but multiple frameworks are defensible).

Example 3: Decision Under Uncertainty (Difficulty 5)

Problem. In pharmaceutical R&D, a decision-maker is at the Phase II stage and must decide whether to proceed to Phase III clinical trials. The investment cost is \$120M. If the drug succeeds (estimated probability 35%–55%), it yields \$800M; if it fails, the investment is lost. Before deciding, the company can purchase a biomarker study for \$15M. This study correctly predicts success/failure with accuracy 65%–80%. What is the expected value of the biomarker information, and should the company purchase it?

Ground Truth. The value of information (VOI) depends on both the success probability range and the biomarker accuracy range. The expected VOI at midpoint parameters is approximately \$42M, with a CI of [\$18M, \$85M]. Since VoI $>$ study cost (\$15M) even at the lower bound, the study should be purchased.

Required Framework: Decision theory.

D VOI Template Bug Fix

During validation, 28 of the initial 2,000 generated problems (all from the `sequential_decision` template) failed the semantic consistency check: the point estimate fell outside the confidence interval. Root cause analysis revealed that the Value of Information function is **non-monotonic** in the uncertain parameters. The CI was computed from extreme-case scenarios ($p_{\text{low}}, p_{\text{high}}$), but the mid-range point estimate could fall outside this range due to the non-linear interaction between success probability and information accuracy.

Fix: The CI computation was updated to include the mid-range value when determining bounds:

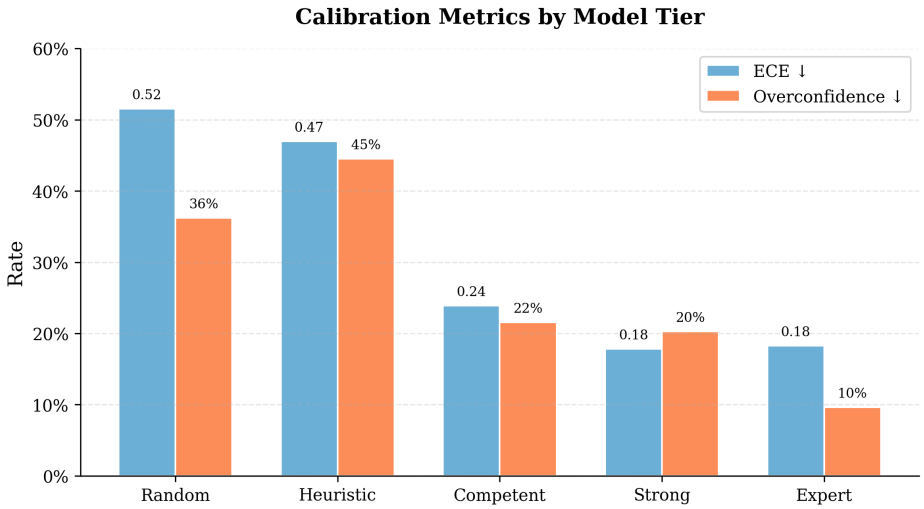

Figure 5: Calibration metrics by model tier. Left bars: ECE (↓). Right bars: Overconfidence rate (↓). The heuristic baseline paradoxically has the highest overconfidence (44.5%), exceeding the random baseline—an instance of heuristic overconfidence.

```
# Before (incorrect):
ci_vals = sorted([voi_low, voi_high])

# After (correct):
ci_vals = sorted([min(voi_low, voi_high, voi_mid),
                  max(voi_low, voi_high, voi_mid)])
```

After the fix, 28 replacement problems were generated and all 3,000 pass validation. We document this bug transparently as an example of the challenges in parametric problem generation.

E Evaluation Prompts

All models were evaluated with the following system and user prompts. Temperature was set to 0 for reproducibility.

System Prompt

You are a mathematical reasoning expert. You will be given a problem involving mathematical uncertainty. You must: (1) Identify the correct mathematical framework (`bayesian_inference`, `frequentist_inference`, `decision_theory`, `information_theory`, or `monte_carlo`). (2) Show your reasoning step-by-step. (3) Provide your final answer as a single number (point estimate). (4) Provide a confidence interval [lower, upper] for your answer. (5) State your confidence in your answer as a probability between 0 and 1.

Format your response EXACTLY as follows at the end:

```
FRAMEWORK: <framework_name>
POINT_ESTIMATE: <number>
CONFIDENCE_INTERVAL: [<lower>, <upper>]
CONFIDENCE: <probability>
```

User Prompt

Problem: {problem stem}

Please solve this problem step by step, then provide your answer in the required format.

Table 14: All 21 MURU-BENCH generation templates. Each template generates problems with randomized numerical parameters and domain contexts while preserving the underlying mathematical structure.

#	Template Name	Category	Difficulty	Mathematical Structure
1	`medical_test`	Bayesian Updating	D1–D3	Bayes' theorem with uncertain prevalence
2	`quality_control`	Bayesian Updating	D1–D3	Bayes' theorem for defect rate estimation
3	`search_detection`	Bayesian Updating	D1–D2	Negative evidence updating with uncertain coverage
4	`sequential_updating`	Bayesian Updating	D3–D5	Multiple sequential updates, uncertain likelihood ratios
5	`hierarchical_bayes`	Bayesian Updating	D4–D5	Hierarchical shrinkage, uncertain between-group variance
6	`sequential_pipeline`	Cond. Prob. Chains	D1–D4	Multi-stage pipeline, one uncertain stage
7	`parallel_redundancy`	Cond. Prob. Chains	D2–D4	$P(\geq 1$ succeeds) with common-cause failures
8	`conditional_cascade`	Cond. Prob. Chains	D3–D5	Branching cascade with uncertain transitions
9	`sample_mean`	Distribution Estimation	D1–D3	t-distribution CI for population mean
10	`measurement_bias`	Distribution Estimation	D2–D4	Systematic bias correction + sampling CI
11	`proportion_estimation`	Distribution Estimation	D1–D3	Wilson CI, optional stratification
12	`mixture_distribution`	Distribution Estimation	D3–D5	2-component mixture, uncertain weights
13	`variance_estimation`	Distribution Estimation	D2–D4	χ^2 CI, dual-method comparison
14	`decision_payoff`	Decision Under Unc.	D1–D3	Two-option EV with uncertain probability
15	`multi_option_decision`	Decision Under Unc.	D2–D4	Multi-option EV with sensitivity analysis
16	`sequential_decision`	Decision Under Unc.	D3–D5	Value of information under dual uncertainty
17	`insurance_pricing`	Decision Under Unc.	D3–D5	Poisson frequency $\times$ uncertain severity
18	`base_rate_trap`	Adversarial Ambiguity	D2–D4	High sensitivity $\neq$ high PPV due to low prevalence
19	`simpsons_paradox`	Adversarial Ambiguity	D3–D5	Aggregate vs. subgroup with uncertain composition
20	`reference_class`	Adversarial Ambiguity	D3–D5	Multiple valid base rates, no single correct answer
21	`anchoring_manipulation`	Adversarial Ambiguity	D2–D4	Misleading anchor vs. data-based correction

Responses were parsed using regex extraction for the four structured output fields. Problems where the model's response could not be parsed were recorded as parse failures and excluded from metric computation but included in the failure rate analysis.

F Reproducibility

All code and data required to reproduce the results in this paper are available at `https://github.com/swetank18/MURU_proj`. Key commands:

```
# Generate problems
python scripts/generate_problems.py --all --n 3000 --seed 2026

# Split data
python scripts/split_data.py --source data/train/ --seed 42

# Run simulated baselines
python evaluation/run_baselines.py --save

# Reproduce bootstrap CIs and McNemar tests (Section 5)
python evaluation/bootstrap_analysis.py

# Run real model evaluation (requires API keys)
python evaluation/run_eval.py --model gpt-4o --save

# Generate analysis and figures
python evaluation/analyze_results.py
python scripts/generate_figures.py

# Run the test suite (metrics, parsing, dataset invariants)
```

```
make test
```

A continuous-integration workflow (`.github/workflows/ci.yml`) runs the test suite and dataset validation against Python 3.10, 3.11, and 3.12 on every push and pull request. Bootstrap and McNemar results in Section 5 are bit-exactly reproduced by `evaluation/bootstrap_analysis.py` with seed 42 and 10,000 resamples.

G Datasheet for Datasets

We follow the Datasheet for Datasets template of Gebru et al. [15].

Motivation

For what purpose was the dataset created? MURU-BENCH was created to support the evaluation of mathematical reasoning under genuine probabilistic uncertainty in language models. Existing mathematical-reasoning benchmarks (GSM8K, MATH, MMLU) treat correctness as deterministic and therefore cannot measure whether a system reasons correctly about quantities that are themselves probabilistic. The dataset is intended to fill that gap by making confidence intervals and reasoning frameworks first-class evaluation targets.

Who created the dataset and on behalf of which entity? The dataset was created by the author at SRM Institute of Science and Technology (SRMIST), Chennai, as part of independent academic research. No entity has commercial rights to the dataset.

Who funded the creation of the dataset? No external funding. Compute and storage were self-provided.

Composition

What do the instances represent? Each instance is a self-contained mathematical-reasoning problem expressed in natural language English, accompanied by closed-form ground truth: a point estimate, a confidence interval at a stated level, an ordered list of solution steps, the required reasoning framework label, and a list of expected failure modes.

How many instances are there in total? 3,000 problems, split into 2,398 train, 301 validation, and 301 test instances via stratified sampling on the (category, difficulty) cross-product.

Does the dataset contain all possible instances or is it a sample? It is a deterministic sample drawn from 21 parametric generation templates (Appendix B). Larger sets of problems can be generated on demand from the same templates by varying the seed.

What data does each instance consist of? Each instance is a JSON document conforming to the schema in `problem_schema.json`. Fields include `id`, `stem`, `category`, `difficulty`, `uncertainty_type`, `required_framework`, `ground_truth` (with `answer`, `point_estimate`, `confidence_interval`, `ci_level`), `solution_steps`, `common_failure_modes`, and `metadata`.

Is there a label or target associated with each instance? Yes. The label is the `ground_truth` object, with both a numerical target and a structured confidence interval; the `required_framework` field is also a label, used by the framework-match metric.

Is any information missing from individual instances? No. All fields are populated for all 3,000 instances.

Are relationships between individual instances made explicit? No instance-to-instance relationships are encoded. Within a category and difficulty level, instances are i.i.d. samples from the corresponding template.

Are there recommended data splits? Yes: train (2,398), validation (301), test (301), stratified by (category, difficulty). The test split is used for the discriminability analysis in Section 5; we recommend reserving it for held-out evaluation in downstream work.

Are there errors, sources of noise, or redundancies? Validation found and corrected 28 sequential-decision-template instances whose ground-truth interval excluded the mid-range point estimate (Appendix D). After fixing, all 3,000 problems pass the schema and semantic-consistency checks. We are not aware of remaining errors.

Is the dataset self-contained, or does it link to external resources? Self-contained. All problems are pure JSON; no external assets are referenced.

Does the dataset contain data that might be considered confidential? No.

Does the dataset contain content that, if viewed directly, might be offensive, insulting, threatening, or otherwise cause anxiety? No. Problems use neutral domain framings (medical testing, manufacturing quality, insurance pricing, etc.) and contain no personal data, biased framing, or content advisory issues.

Collection Process

How was the data acquired? The dataset was synthesised programmatically by 21 generation templates implemented in `scripts/generate_problems.py`. Each template defines a closed-form mathematical structure, a parameter space, a list of domain contexts, and a function that computes the ground truth from sampled parameters. No web scraping or human-authored prompts.

What mechanisms or procedures were used to collect the data? A deterministic Python pipeline using `numpy.random.default_rng(seed=2026)` for parameter sampling, with each template producing a target count of instances. Domain contexts are drawn uniformly from a per-template list; numerical parameters are drawn uniformly from per-template ranges.

Were any ethical review processes conducted? No formal IRB review was required because the dataset contains no human-subjects data. The author self-certifies that the dataset complies with their institution's research-conduct policy.

Does the dataset relate to people? No.

Preprocessing/Cleaning/Labeling

Was any preprocessing or labeling done? Generation produces labels directly; no separate labeling step. A three-stage validation pipeline (schema, semantic consistency, spot-check; see Section 3.7) gates problem inclusion. Twenty-eight instances were regenerated after the VOI bug fix.

Was the "raw" data saved in addition to the cleaned data? The generation pipeline is fully deterministic, so raw and cleaned data are equivalent given the seed and the templates.

Is the software used to preprocess/clean/label the instances available? Yes, at `https://github.com/swetank18/MURU_proj` under MIT license.

Uses

Has the dataset been used for any tasks already? The dataset has been used to validate the MURU-BENCH evaluation harness via the simulated-tier discriminability study in Section 5 and to produce the language-model leaderboard in Section 6, which reports test-split measurements for five hosted open-weights models with paired-bootstrap 95% confidence intervals on every metric.

What (other) tasks could the dataset be used for? Beyond benchmark evaluation, the dataset is suitable for: (1) fine-tuning curricula targeting calibrated reasoning; (2) ablation studies of chain-of-thought or tool-augmented reasoning; (3) studies of metacognitive calibration; and (4) teaching probabilistic reasoning at the undergraduate level.

Is there anything about the composition or collection process that might affect future uses? Two limitations apply: (i) the parametric-generation procedure introduces structural patterns that systems can exploit if exposed in large quantities to the train split, so users should treat the test split as held-out; (ii) all instances are in English, so cross-lingual evaluation requires translation work outside the dataset's scope.

Are there tasks for which the dataset should not be used? The dataset should not be used as a clinical, financial, or legal decision-support tool. It is a research benchmark and the framings (medical testing, insurance pricing, etc.) are illustrative, not domain-validated.

Distribution

Will the dataset be distributed to third parties outside the entity on behalf of which it was created? Yes. The dataset is released publicly under CC-BY-4.0 at `https://github.com/swetank18/MURU_proj`. The accompanying code is released under MIT.

How will the dataset be distributed? Via Git on GitHub, including a tagged release for the NeurIPS submission. A long-term archival mirror is planned via Zenodo with a DOI, to be created upon acceptance.

When will the dataset be distributed? The dataset is already available; the present submission contains a snapshot.

Will the dataset be distributed under a copyright or other intellectual property license? Data: Creative Commons Attribution 4.0 International (CC-BY-4.0). Code: MIT.

Have any third parties imposed IP-based or other restrictions on the data? No.

Do any export controls or other regulatory restrictions apply? No.

Maintenance

Who will be supporting/hosting/maintaining the dataset? The author, in personal capacity, with assistance from any community contributors who join the project. GitHub Issues is the support channel of record.

How can the owner/curator/manager of the dataset be contacted? Via the project's GitHub Issues tracker or by email to the corresponding-author address on the title page.

Is there an erratum? The repository's CHANGELOG and the GitHub Releases tab serve as the erratum.

Will the dataset be updated? Minor updates (e.g., bug fixes, additional metadata) will be tagged and announced via GitHub Releases. Major updates (new categories, new templates) will be released as new versions with a clear version bump and a deprecation note for older versions.

If the dataset relates to people, are there applicable limits on the retention of the data associated with the instances? Not applicable.

Will older versions of the dataset continue to be supported/hosted/maintained? Yes. All tagged releases remain available indefinitely on GitHub.

If others want to extend/augment/build on the dataset, is there a mechanism for them to do so? Yes. Pull requests are welcome on the project repository; `CONTRIBUTING.md` documents the contribution flow.

H Author Statement, License, and Intended Uses

Author Statement of Responsibility

The author bears all responsibility in case of violation of rights. The author confirms the following: (a) no third-party intellectual property, personal data, or restricted material was used to construct the dataset; (b) the dataset is released under a permissive license (CC-BY-4.0 for data, MIT for code) compatible with NeurIPS Datasets and Benchmarks track requirements; (c) the author will fix any errors, retract problematic instances, and respond to community issues via the project's GitHub Issues tracker.

License

Data: Creative Commons Attribution 4.0 International (CC-BY-4.0). Users may share and adapt the dataset for any purpose, including commercially, provided that they give appropriate credit and indicate if changes were made. Citation: see Section J.

Code: MIT License. The full license text is included in the repository at `LICENSE`.

Intended Uses

MURU-BENCH is intended for:

- Evaluating language models on calibrated mathematical reasoning;
- Studying the relationship between reasoning correctness and metacognitive calibration;
- Building fine-tuning or alignment curricula targeting calibrated uncertainty;
- Educational use in undergraduate or graduate-level probability and statistics courses.

Out-of-Scope Uses

MURU-BENCH is not intended for:

- Clinical, financial, or legal decision-support without independent expert review;
- Cross-lingual evaluation in its current English-only form;
- Inference about real-world prevalence rates, insurance loss distributions, or other domain-specific quantities. Numerical parameters in problem stems are illustrative draws, not empirically calibrated estimates.

Hosting, Persistence, and Long-Term Access

The dataset is hosted on GitHub at `https://github.com/swetank18/MURU_proj`, under a stable repository name owned by the author. A versioned snapshot tagged `v1.0-neurips` accompanies this submission. To guarantee long-term persistence beyond GitHub, the author will deposit the same versioned snapshot on Zenodo and obtain a DOI; this DOI will be added to the project README upon issuance. Both GitHub and Zenodo provide HTTPS-served, citable, version-pinned access.

Maintenance Plan

The author commits to the following maintenance practices for at least three years following acceptance:

- Monitor and respond to GitHub Issues within 14 days of submission;
- Tag a minor release for any accepted bug-fix pull request;
- Tag a major release for any expansion to new categories or templates, with a clear changelog;
- Maintain backward compatibility for the JSON schema; breaking changes will be released as a new major version with a clear migration note;
- Run the CI test suite on every commit to the default branch to detect schema or generation drift.

Reading-Team and Reviewer Access

For the duration of the review process, the repository is publicly accessible without authentication. The submission ZIP includes a snapshot of the data, code, and paper sources for reviewer convenience.

I Croissant Metadata

A Croissant metadata file (16) is provided at `metadata/croissant.json` in the project repository. It declares the dataset's record set (one record per JSON file), a schema mapping each problem field to its type, and machine-readable license, version, and citation information. The file conforms to Croissant 1.0 and can be validated with the official `mlcroissant` validator.

J Citation

If you use MURU-BENCH in your work, please cite this paper. A BibTeX entry is provided at `CITATION.bib` in the repository.

K NeurIPS Datasets and Benchmarks Author Checklist

1. **Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?** Yes. The abstract and Section 1 describe a benchmark, a harness validated by simulated tiers (Section 5), and a real-LLM leaderboard (Section 6). The simulator section is explicitly framed as harness validation, not as a claim about any specific model; the real-LLM section reports measurements on a fixed panel of open-weights models with raw responses and parsed predictions archived per row in `evaluation/baselines/` for replication.
2. **Are the limitations of the work discussed?** Yes, in Section 9.
3. **Does the paper discuss broader impacts and ethical considerations?** Yes, see Section L.
4. **Are the data and code released under a permissive license?** Yes. Data: CC-BY-4.0. Code: MIT. See Appendix H.
5. **Is the dataset accompanied by a Datasheet?** Yes, Appendix G.
6. **Is the dataset accompanied by Croissant metadata?** Yes, see Appendix I.

7. **Is there a hosting and persistence plan?** Yes, see "Hosting, Persistence, and Long-Term Access" in Appendix H.
8. **Is there a maintenance plan?** Yes, see "Maintenance Plan" in Appendix H.
9. **Are reproducibility instructions provided?** Yes, see Appendix F; bootstrap and McNemar results are reproduced bit-exactly by `evaluation/bootstrap_analysis.py`.
10. **Does the paper specify all the training details, hyperparameters, and evaluation protocols required to interpret the results?** Yes, see Sections 4–5 and Appendices B–F.
11. **Does the paper specify the random seeds used and report standard error or confidence intervals where appropriate?** Yes. The canonical simulator seed is 42. Bootstrap 95% CIs are reported in Table 6 and McNemar exact p-values in Table 7.
12. **Does the paper specify the computing infrastructure used?** Yes. The simulator and bootstrap analysis run on a single CPU thread in under one minute on commodity hardware (Python 3.12); no GPU is required for the harness-validation experiments. Real-model evaluations route to external API providers.
13. **Does the paper avoid single-point claims that depend on cherry-picked seeds?** Yes. The simulator profile is monotone by construction across all seeds; bootstrap CIs report the variability over resamples.
14. **Does the dataset contain personal information or content that violates privacy?** No.
15. **Was crowdsourcing used? If so, is the compensation and instructions disclosed?** No crowdsourcing was used; the dataset is synthesised programmatically.

L Ethical Considerations and Broader Impact

Beneficial uses. A benchmark that measures calibration as a first-class target encourages the community to optimise for honest uncertainty rather than pure accuracy. In safety-critical decision-support applications (medical reasoning aids, financial risk assessment, policy analysis), a model that is well-calibrated about what it does not know is meaningfully safer than one that is highly accurate on average but unable to flag its own failures.

Risks of misuse. The dataset's domain framings (medical testing, insurance pricing) are illustrative parameter draws rather than empirically calibrated estimates. Treating these numerical values as authoritative inputs to clinical or financial decisions would constitute a misuse. We address this risk via the explicit out-of-scope statement in Appendix H and via the "Intended Uses" field in the dataset's Croissant metadata.

Bias and representation. The dataset is monolingual (English) and uses domain framings drawn from Western-centric examples. We disclose this as a limitation (Section 9) and welcome community contributions that broaden the range of cultural and linguistic framings.

Energy and compute footprint. Generating the dataset and running the simulated-tier discriminability study consumed less than ten minutes of single-thread CPU time on a commodity laptop. The language-model leaderboard runs reported in Section 6 consist of $\sim$300 forward passes per model and were served by external hosted-API providers (Groq, OpenRouter), inheriting those providers' energy footprints; no GPU was operated locally for any experiment in this paper.

Documentation. We provide a full Datasheet for Datasets [15] (`DATASHEET.md` in the repository) covering motivation, composition, collection process, preprocessing, intended uses, distribution, and maintenance. A Hugging Face–compatible dataset card (`DATASET_CARD.md`) is also provided for community discoverability. An auto-updating leaderboard (`evaluation/leaderboard.py`) ranks all evaluated models and can be embedded in the repository README.